A Large-Scale Analysis of AI-Assisted Pull Requests: Observations, Biases, and Limits of Behavioral Inference

Ahmed Mursal
School of Computing
Edinburgh Napier University
Edinburgh, Scotland
40646515@live.napier.ac.uk

***Abstract*—Through this analysis, the paper examines whether large-scale observational pull request datasets labeled with AI agent information can, in practice, support valid behavioral inference about AI coding agents. By tracing the full analytical trajectory—from initially compelling statistical signals to empirical validation and structural diagnosis—the study identifies the specific dataset properties that undermine inference, including extreme contribution inequality, automation and single-user dominance, attribution ambiguity, non-independence, measurement error in keyword-based detection, and widespread data loss. The work further evaluates the reliability of common metadata-driven proxies for testing behavior and demonstrates that, without manual validation, such proxies systematically overestimate test inclusion. While these limitations preclude defensible agent-level behavioral claims, the analysis shows that certain structural and distributional properties of the dataset remain empirically interpretable. Based on these findings, the paper distills methodological guidance for future research, emphasizing validation-first study design, explicit measurement error quantification, and dataset construction practices that are necessary for reliable empirical analysis of AI-assisted software development.**

I. Introduction

AI-powered coding assistants are now embedded in everyday software development workflows. Tools such as GitHub Copilot, OpenAI Codex, Cursor, Devin, and Claude Code are used by millions of developers and are increasingly credited with accelerating development, improving productivity, and influencing software quality practices. As these tools become pervasive, understanding how they shape real-world development behavior—particularly testing practices—has become an important empirical question for both researchers and practitioners.

Recent software engineering research has responded to this need by leveraging large-scale datasets derived from development platforms such as GitHub. These datasets appear especially attractive: they capture real production activity at unprecedented scale, promise high statistical power, and seemingly allow direct comparison of AI agent behavior "in the wild". In particular, pull request (PR) datasets labeled with AI agent information have been used to make claims about differences in testing behavior, development style, and quality practices across tools.

This study began with exactly that motivation. Our original goal was to determine whether different AI coding agents exhibit systematically different testing behaviors. Using a large PR dataset with agent labels, we initially observed striking patterns: some agents appeared to include tests in nearly all PRs, while others rarely did so. Standard statistical analysis suggested large, statistically significant differences, seemingly supporting the narrative that AI agents differ meaningfully in how they support testing.

However, as the analysis progressed, these early findings raised a more fundamental question: what exactly do these apparent differences represent?

Closer inspection of the data revealed unexpected and troubling characteristics. Contribution activity was extremely concentrated, with a small number of users responsible for a disproportionate fraction of PRs. Some agent categories were dominated by single accounts producing thousands of PRs. A substantial portion of PRs were no longer accessible, preventing direct verification of their contents. Manual inspection further showed that keyword-based indicators of testing often failed to correspond to actual test files or meaningful test quality.

These observations forced a reassessment of the original research question. Rather than asking whether AI agents behave differently, this work reframes the problem: can such PR datasets, as currently constructed, support valid agent behavior analysis at all?

In response, this paper presents a structured empirical investigation that combines large-scale quantitative analysis with targeted manual validation. We document what the dataset clearly shows—such as adoption patterns and extreme contribution inequality—while also identifying where inference breaks down due to attribution ambiguity, measurement error, automation, and missing data. Rather than advancing unsupported behavioral claims, this study offers a methodological account of how seemingly compelling results emerge, why they are fragile, and how future research can avoid similar pitfalls.

By telling the full analytical story—from initial hypothesis, through failure, to methodological insight—this work

contributes both empirical observations and a cautionary framework for AI tool research based on large observational datasets.

A. Research Questions

This study investigates whether large-scale observational pull request datasets labeled with AI agent information can support valid agent behavior claims. Specifically, it examines (i) whether such datasets satisfy the assumptions required for agent-level behavioral comparison, (ii) which structural and measurement properties undermine inference, (iii) the reliability of metadata- and keyword-based proxies for testing behavior, and (iv) which empirical properties remain valid despite these limitations. Based on these findings, the study derives methodological guidance for future empirical research on AI-assisted software development.

II. Dataset Analysis

A. Dataset Source

This study uses the AIDev dataset released as part of the MSR 2026 Mining Challenge. The dataset was collected by Li et al. and made publicly available to support research on AI-assisted software development. It aggregates GitHub pull request data annotated with AI agent attribution labels, representing one of the largest publicly available datasets of AI-assisted development activity.

The dataset was obtained directly from the official MSR 2026 challenge release and was used without modification to its original labels or structure, except where explicitly noted for analysis and validation purposes.

B. Dataset Scope and Composition

The full dataset contains 932,791 pull requests, 72,189 unique users, and five AI agent labels (OpenAI Codex, GitHub Copilot, Cursor, Devin, and Claude Code). Each pull request record includes an agent label, user identifier, repository information, pull request title and description, timestamps and merge status, and links to the original GitHub PR (when available).

The dataset spans a large number of repositories, programming languages, and development contexts, providing broad coverage of real-world AI-assisted development activity.

C. Intended Use and Initial Assumptions

The dataset is explicitly designed to support observational analysis of AI-assisted development practices. At face value, its scale and breadth appear well suited for behavioral comparison across agents, including analysis of testing practices via PR metadata.

However, as this study demonstrates, the suitability of the dataset for such inference depends critically on assumptions about attribution accuracy, data completeness, user independence, and measurement validity. These assumptions are examined empirically rather than taken for granted.

TABLE I
Dataset Summary

Metric	Value
Total PRs	932,791
Unique users	72,189
Gini coefficient	0.83
Inaccessible PRs in manual review	238
Agents	5

III. Results

This section reports empirical findings derived from the dataset and from manual validation. Results are presented descriptively and diagnostically rather than as behavioral conclusions about AI agents. Where statistical measures are reported, they are used to characterize distributional properties and structural patterns, not to infer causality or agent intent.

A. Dataset-Level Structural Characteristics

Analysis of the full dataset (932,791 pull requests) reveals extreme structural imbalance in contribution activity. Contribution inequality is severe, with the top 1% of contributors responsible for over 40% of all PRs. The computed Gini coefficient (≈ 0.83) indicates an exceptionally concentrated distribution. This concentration is not uniform across agents, with certain categories dominated by single users producing thousands of PRs.

B. Accessibility and Data Loss

A manual review of 385 pull requests revealed that 238 were inaccessible (404 errors), especially prevalent among high-volume users. This loss limits verification and distorts sampling, as accessibility is not uniformly distributed.

C. Manual Validation of Test Presence and Quality

A stratified manual review of 385 pull requests (77 per agent) established ground truth for test presence and quality using a four-level rubric. Testing practices are heterogeneous, ranging from extensive to minimal. Keyword-based detection is unreliable, with false positives from templates and false negatives when tests are not explicitly described.

D. Measurement Error in Automated Detection

Bidirectional validation shows that automated test detection systematically overestimates test presence, varying by language and repository conventions. This reflects inherent limitations in PR-level metadata.

E. Debiasing Effects on Aggregate Patterns

Applying user-level filtering alters aggregate adoption statistics, shifting agent shares and increasing distributional diversity. Raw PR counts are unstable indicators of adoption or behavior.

F. Summary of Empirical Findings

In summary, the results show extreme contribution concentration, significant data loss, heterogeneous testing practices, error-prone detection, and distorted aggregate statistics. These describe dataset properties rather than agent behaviors. For example, initial chi-square tests comparing keyword-detected test presence across agents yielded statistically significant differences (p ¡ 0.001), but these results were invalidated after accounting for user-level dependence and measurement error.

IV. Threats to Validity

This section outlines threats to validity following standard empirical software engineering practice.

A. Internal Validity

Internal validity is threatened by attribution ambiguity, as agent labels indicate association, not authorship. Human modification and automation contamination complicate interpretation. Manual verification involves subjective judgment.

B. Construct Validity

Construct validity is limited by operationalization of testing behavior, focusing on presence and quality rather than effectiveness. Keyword-based detection exhibits false positives and negatives, mitigated but not eliminated by manual validation.

C. External Validity

External validity is constrained by the dataset's GitHub scope and temporal snapshot. AI tools evolve rapidly, and inaccessible PRs introduce uncertainty about representativeness.

D. Conclusion Validity

Conclusion validity is threatened by structural dependence, violating independence assumptions. Descriptive statistics remain valid, but inferential statistics cannot support behavioral differences.

V. Impact and Implications

This work impacts empirical research practice, dataset construction, and industrial interpretation of AI tool studies. Its contribution lies in establishing what can and cannot be responsibly inferred from large PR datasets.

A. Impact on Empirical Software Engineering Research

The primary impact is methodological. It demonstrates how apparently strong results can emerge from observational datasets even when core assumptions are violated. By documenting the analytical trajectory, this work provides a cautionary framework for interpreting future AI-assisted programming studies.

B. Impact on Dataset Design and Curation

The analysis reveals structural properties affecting inference, including contribution inequality, automation contamination, and accessibility loss. This provides guidance for future dataset construction, emphasizing persistent identifiers and archival preservation.

C. Impact on Industrial and Practitioner Interpretation

From an industry perspective, this work cautions against drawing operational conclusions from aggregate PR statistics. Apparent differences may reflect dataset artifacts rather than intrinsic tool characteristics.

D. Broader Research Implications

This study argues for treating methodological failure as a valid research outcome, contributing to a more transparent empirical research culture.

VI. Related Work and Methodological Foundations

This work draws on empirical software engineering methodology, large-scale repository datasets, and AI-assisted programming studies.

A. Empirical Software Engineering Methodology and Validity

Foundational work emphasizes valid measurement and sampling [1], [2]. Sampling practices follow precedents like Naghashzadeh et al. [3].

B. Large-Scale Observational Datasets and Repository Mining

Repository mining research highlights metadata limitations and incompleteness [4], [5], motivating accessibility checks and manual verification.

C. AI-Assisted Programming and Coding Agents

Recent studies examine AI tools' role in development [6], [7], [8], but rely on controlled experiments rather than large observational data.

D. Measurement, Testing Practices, and Statistical Foundations

Testing practices are studied via textual proxies [9], [10]. Statistical guidance emphasizes effect size over significance [11], [12].

E. Positioning of the Present Work

This study contributes methodological analysis of AI-assisted PR datasets, clarifying valid inference limits.

VII. Conclusion

This paper demonstrates why certain behavioral questions about AI coding agents cannot be answered using such PR datasets, providing empirical techniques for detecting invalid inference.

Initial analysis revealed apparent differences in testing practices across tools, but deeper investigation showed these patterns do not support valid behavioral conclusions due to attribution ambiguity, structural dependence, and measurement limitations.

The methodological contribution lies in establishing baseline validation techniques for AI-assisted development datasets, emphasizing empirical verification over assumption-driven analysis. Future work should focus on instrumented studies and longitudinal designs to enable reliable agent behavior analysis.

REFERENCES

[1] C. Wohlin, P. Runeson, M. H"ost, M. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.

[2] B. Kitchenham, S. L. Pfleeger, L. Pickard, P. W. Jones, D. C. Hoaglin, K. E. Emam, and J. Rosenberg, "Preliminary guidelines for empirical research in software engineering," *IEEE Transactions on Software Engineering*, vol. 28, no. 8, pp. 721–734, 2002.

[3] A. Naghashzadeh *et al.*, "Developer behavior in q&a ecosystems: A large-scale empirical study," in *Proceedings of the 44th International Conference on Software Engineering*. ACM, 2022.

[4] S. Baltes, L. Dumani, C. Treude, and S. Diehl, "Sotorrent: Reconstructing and analyzing the evolution of stack overflow posts," in *Proceedings of the 15th International Conference on Mining Software Repositories*. ACM, 2018, pp. 319–330.

[5] M. Asaduzzaman, A. Mashiyat, C. K. Roy, and K. A. Schneider, "Answering questions about unanswered questions of stack overflow," in *Proceedings of the 10th Working Conference on Mining Software Repositories*. IEEE, 2013, pp. 97–100.

[6] P. Vaithilingam *et al.*, "Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models," in *Proceedings of the 44th International Conference on Software Engineering*. ACM, 2022.

[7] J. Finnie-Ansley *et al.*, "Robots for the masses: A survey of code generation tools powered by large language models," in *Proceedings of the 44th International Conference on Software Engineering*. ACM, 2022.

[8] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[9] F. Calefato, F. Lanubile, and N. Novielli, "Ask me anything: A study of stack overflow questions related to software testing," in *Proceedings of the 40th International Conference on Software Engineering*. ACM, 2018, pp. 886–896.

[10] H. Mi *et al.*, "An empirical study of the characteristics of popular stack overflow questions," in *Proceedings of the 24th Asia-Pacific Software Engineering Conference*. IEEE, 2017, pp. 144–153.

[11] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*. Routledge, 1988.

[12] A. Agresti, *An Introduction to Categorical Data Analysis*. Wiley, 2018.